ESCUELA POLITÉCNICA NACIONAL
Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE LABORATORIO · SEMANA 3

Clasificación de variables y población vs. muestra

Laboratorio complementario · Semana 3
Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	Romo Navas María Paula · usuario de GitHub: paularomo2006
Docente	Christian Suárez
Período académico	2026-B
Fecha de entrega	23/10/2026
Repositorio (fork)	https://github.com/paularomo2006/IDSD422-lab-muestreo-poblacion

1. Introducción

RetailAndina S.A. necesita conocer mejor la información de sus clientes para realizar análisis confiables y tomar decisiones comerciales basadas en los datos. Primeramente, se tiene que identificar correctamente las variables de su base de fidelización, puesto que no todos los datos se analizan de igual forma. También es importante comprender cómo obtener información representativa de una población de 400 clientes utilizando muestras de 20 registros. En esta práctica trabajé con Python y R para clasificar las variables y estudiar diferentes métodos de muestreo, considerando que una muestra mal seleccionada puede producir resultados poco confiables, independientemente de su tamaño.

2. Objetivos

Objetivo General

- Clasificar las variables de la base de clientes de RetailAndina S.A. y analizar tres métodos de muestreo para comprender su influencia en la estimación del ingreso mensual promedio.

Objetivos Específicos

- Clasificar las variables del archivo clientes dependiendo de sus características numéricas o categóricas.
- Verificar mediante los lenguajes Python y R los tipos de datos, los valores únicos y el orden de las variables ordinales.
- Comparar los métodos de muestreo por conveniencia, aleatorio simple y estratificado utilizando muestras de 20 clientes.
- Interpretar las diferencias entre las estimaciones muestrales y el parámetro poblacional para reconocer posibles sesgos y variaciones.

3. Desarrollo de la práctica

3.1 Parte A: Clasificación de las variables

Variable	Tipo	Justificación	Verificación en código
cliente_id	Categórica nominal	Identifica al cliente, contiene números y letras.	str/chr
edad	Numérica discreta	Se registró como años enteros, no con meses ni días.	int64/int
numero_hijos	Numérica discreta	Número de hijos solo puede ser entero.	int64/int

genero	Categórica nominal	Las categorías no tienen orden que sea indispensable. No es binomial porque existe la opción Otro.	str/chr
nivel_educacion	Categórica ordinal	Los niveles de educación tienen un orden.	str/chr
ciudad	Categórica nominal	Puede haber varios registros de distintas ciudades.	str/chr
ingresos_mensuales	Numérica continua	Una cantidad de dinero puede tener valores decimales y/o enteros.	int64/int
satisfaccion	Categórica ordinal	Es una escala de 1-5, tiene un orden específico.	int64/int

P1. ¿Qué te dijo el dtype de satisfaccion y por qué no basta para clasificarla? ¿Qué pasa si una ordinal queda como texto sin declarar el orden?

El tipo de dato indicó que la variable **satisfaccion** era numérica (int64/int), pero esto no significa que sea una variable cuantitativa. En realidad, es ordinal porque sus valores representan niveles de satisfacción o una escala, donde 4 no necesariamente significa el doble de satisfacción que 2, eso depende de cómo se definido cada nivel. Si una variable ordinal queda almacenada como texto sin declarar su orden, el programa puede organizar sus categorías alfabéticamente en lugar de seguir su jerarquía real. Por eso declaré el orden de sus categorías en Python y R.

3.2 Parte B: Población vs. Muestra

Población = 400 clientes (Quito=200. Guayaquil=120, Cuenca=80)

La variable de interés fue el ingreso mensual, por lo que el propósito consistió en estimar su promedio mediante muestras de 20 clientes. Para lo que se planteó los siguientes métodos de muestreo: conveniencia, aleatorio simple y estratificado. Según el método pude estudiar cómo la selección de la muestra influye en las estimaciones y en la representatividad de cada ciudad.

Concepto	Valor
Población (N)	400 clientes
Variable de interés	ingresos_mensuales
Parámetro poblacional (media)	1330.72 USD
¿Por qué se muestrea si aquí el parámetro existe y se calculó?	Porque no siempre se podrá calcular el parámetro de toda la población, entonces en la mayoría de los casos es necesario tomar una muestra para poder calcularlo. Para lo que se debe aplicar el método de muestreo que más se adecúe a la situación. En este caso se conoce toda la población y se puede obtener la media, que sirve como un valor de referencia.

3.3 Comparación de modelos de muestreo

Python

Diseño	n	Media estimada	Composición por ciudad	Diferencia vs. parámetro
Conveniencia (head)	20	1348.50	Quito = 20	+17.78
Aleatorio simple (sample)	20	1336.00	Cuenca = 8, Quito = 8, Guayaquil = 4	+5.28
Estratificado (groupby + 5%)	20	1287.50	Quito = 10 Guayaquil = 6 Cuenca = 4	-43.22

R

Diseño	n	Media estimada	Composición por ciudad	Diferencia vs. parámetro
Conveniencia (head)	20	1348.50	Quito = 20	+17.78
Aleatorio simple (sample)	20	1318.00	Cuenca = 5, Quito =8 , Guayaquil = 7	-12.72
Estratificado (groupby + 5%)	20	1226.00	Quito = 10 Guayaquil = 6 Cuenca = 4	-104.72

Medias de 5 sorteos distintos del aleatorio simple (semillas 1 a 5)

Python

Semilla	1	2	3	4	5
Media	1336.00	1405.00	1235.00	1491.50	1288.00

R

Semilla	1	2	3	4	5
Media	1318.00	1287.00	1233.00	1385.50	1283.00

P2. ¿Cuánto vale el parámetro poblacional y cuánto estimó cada diseño? ¿Cuál quedó más cerca y por qué eso no alcanza para declarar un ganador?

El parámetro es de 1330.72 y los tres diseños estimaron valores similares, igualmente dependiendo de cada lenguaje (Python y R). En ambos lenguajes el diseño que más se acercó fue el **Aleatorio simple**. Sin embargo, obtener un resultado aproximado al parámetro real en una sola ejecución no garantiza que ese método sea el más confiable, porque la proximidad puede estar relacionada con el azar. Para evaluar correctamente a un diseño se debe considerar el cómo se selecciona los registros.

P3. La muestra por conveniencia también tiene 20 filas. ¿Qué defecto tiene y cómo lo delatas en la composición por ciudad? ¿Se corrige tomando más filas?

El defecto que tiene el muestreo por conveniencia, a pesar de tener el mismo número de registros, es que la selección de los registros no es equitativa. Los clientes fueron seleccionados únicamente de la ciudad de Quito, por lo que las otras ciudades quedan excluidas. Aumentar la cantidad de registros no soluciona necesariamente el problema, porque si la selección continúa siguiendo el orden del archivo, el sesgo puede mantenerse.

P4. El sorteo aleatorio dio un número distinto en cada semilla. ¿Qué implica esa variabilidad para una empresa que va a decidir con un solo estudio?

Que siempre es necesario realizar varios estudios para poder verificar que el método sea más certero. Las estimaciones pueden variar dependiendo de los clientes que se seleccionan, incluso si se uso el mismo método y tamaño de muestra.

P5. ¿Cuándo usarías cada diseño en la práctica? Relaciona cada uno con un uso real.

Utilizaría el muestreo por conveniencia cuando necesitara realizar una prueba rápida o verificar que un procedimiento funciona, pero no para sacar conclusiones sobre todos los clientes. Aplicaría el muestreo aleatorio simple cuando quisiera obtener una muestra seleccionada al azar de toda la población. Finalmente, elegiría el muestreo estratificado cuando necesitara garantizar la

participación de clientes de diferentes ciudades, especialmente si sus características pudieran variar entre ellas.

4. ¿Qué aprendí con esta práctica?

Esta práctica me ayudó a comprender que el tipo de dato que identifica un lenguaje de programación no siempre corresponde a su clasificación estadística. Un ejemplo fue la variable satisfacción, que inicialmente aparecía como numérica, pero debía considerarse como categórica ordinal porque representa niveles y no cantidades proporcionales.
También, comprendí que para obtener un parámetro que sea más aproximado a la realidad, debo identificar el método de muestreo más adecuado, dependiendo de lo que necesito hacer. Es importante realizar varias repeticiones o sorteos para comprobar que el resultado sea representativo.

5. Conclusiones

- La clasificación de las variables permitió identificar correctamente los datos de los 12 clientes y sus ocho columnas. Comprobé que la satisfacción es una variable categórica ordinal, aunque se almacene como número, y que el nivel educativo debe conservar un orden específico.

- El estudio de muestreo permitió reconocer que una población de 400 clientes no necesariamente genera un resultado representativo de todos los registros. En particular, seleccionar únicamente los primeros 20 clientes excluye a Guayaquil y Cuenca, lo que muestra un problema de representación que no se resuelve aumentando el tamaño de la muestra.

- Para RetailAndina S.A., recomendaría utilizar métodos probabilísticos cuando se necesiten resultados que representen a toda la población. El muestreo estratificado resulta especialmente útil cuando interesa asegurar la participación de las tres ciudades. En conclusión, las estimaciones deben estar precedidas por la buena elección de un método y un tamaño de muestra que permita obtener resultados representativos, no necesariamente aumentarlo.

Declaración de Inteligencia Artificial

Utilicé ChatGPT de OpenAI como herramienta de apoyo para resolver dudas puntuales de sintaxis en Python y R, corregir errores de ejecución y aclarar conceptos relacionados con la clasificación de variables. La revisión de los resultados y las comprobaciones del laboratorio se realizaron mediante las ejecuciones del código. Además, me ayudó a mejorar la redacción y ortografía del informe.

Referencias

- Pandas development team. (s. f.). pandas.CategoricalDtype. https://pandas.pydata.org/docs/reference/api/pandas.CategoricalDtype.html
- Gabinete de Estudios. (s. f.). Qué son las técnicas de muestreo, tipos y características: Todo en esta guía. https://gabinetedeestudios.com/tecnicas-de-muestreo/
- R Core Team. (s. f.). Factors: R documentation. R Foundation for Statistical Computing. https://stat.ethz.ch/R-manual/R-devel/library/base/html/factor.html
- Escuela Politécnica Nacional. (2026). Laboratorio: Clasificación de variables y población vs. muestra [Repositorio académico de la asignatura IDSD422]. https://github.com/christian-suarez1979/IDSD422-lab-muestreo-poblacion

Anexos

R

```r
# ------------------------------------------------------------------------
# 2. Muestra por conveniencia: "los primeros que llegaron"
# ------------------------------------------------------------------------
# TODO 1: toma como muestra las primeras 20 filas de la población.
#         Pista: head(poblacion, 20) — igual que la demo de clase.
#         Después de correrlo, mira la composición por ciudad que imprime el
#         reporte: ¿es un corte al azar? ¿alguna ciudad falta por completo?
muestra_conv <- head(poblacion, 20)
```

Figura 1. Implementación del muestreo por conveniencia en R mediante la selección de los primeros 20 registros.

```r
# ------------------------------------------------------------------------
# 3. Muestra aleatoria simple: el sorteo de 20
# ------------------------------------------------------------------------
# TODO 2: sortea 20 clientes con la misma probabilidad para todos (20/400 = 5 %).
#         Pista: poblacion[sample(nrow(poblacion), 20), ] con set.seed(1)
#         delante para que sea reproducible (sin set.seed cambia cada vez).
# TODO 3: repite el sorteo con las semillas 2 a 5 y anota las 5 medias en el
#         informe: la variabilidad de la estimación es parte de la lección.
#         Pista: for (semilla in 1:5) { set.seed(semilla); ... }
set.seed(1)
muestra_aas <- poblacion[sample(nrow(poblacion), 20), ]
```

Figura 2. Implementación del muestreo aleatorio simple en R utilizando una semilla para obtener resultados reproducibles.

```r
# ------------------------------------------------------------------------
# 4. Muestra estratificada: 5 % dentro de cada ciudad
# ------------------------------------------------------------------------
# TODO 4: muestrea dentro de cada ciudad (el estrato), con afijación
#         proporcional: 5 % de cada una → 10 de Quito, 6 de Guayaquil y 4 de
#         Cuenca (20 en total).
#         Pista: library(dplyr); poblacion |> group_by(ciudad) |> slice_sample(prop = 0.05)
#         ¿Qué garantiza este diseño que el sorteo simple no garantiza?
library(dplyr)
set.seed(1)
muestra_est <- poblacion |> group_by(ciudad) |> slice_sample(prop = 0.05)
```

Figura 3. Aplicación del muestreo estratificado en R con la biblioteca dplyr, seleccionando el 5 % de cada ciudad.

```
Población: 400 clientes × 8 columnas
Parámetro poblacional — ingreso medio: 1330.72 USD
Ciudades en la población:
   Cuenca Guayaquil     Quito
       80       120       200
[Conveniencia] n=20  media=1348.50 USD  (parámetro 1330.72, diferencia +17.78)
    composición por ciudad:
Quito
   20
[Aleatorio simple] n=20  media=1318.00 USD  (parámetro 1330.72, diferencia -12.72)
    composición por ciudad:
   Cuenca Guayaquil     Quito
        5         7         8
[Estratificado] n=20  media=1226.00 USD  (parámetro 1330.72, diferencia -104.72)
    composición por ciudad:
   Cuenca Guayaquil     Quito
        4         6        10
Parámetro poblacional: 1330.72 USD
```

Figura 4. Resultados de los tres métodos de muestreo en R, incluyendo las medias estimadas y la distribución de clientes por ciudad.

```
                          diseno  n media_estimada
             Conveniencia (head) 20         1348.5
       Aleatorio simple (sample) 20         1318.0
 Estratificado (slice_sample) 20         1226.0
Con estas salidas responde en informe.md las preguntas P2 a P5:
  P2: ¿cuál diseño quedó más cerca del parámetro y por qué un solo
      número no basta para declarar un ganador?
  P3: ¿qué defecto tiene la muestra por conveniencia y cómo lo delata
      su composición por ciudad? ¿Se corrige con más filas?
  P4: ¿qué implica la variabilidad entre semillas para una decisión
      tomada con un solo estudio?
  P5: ¿cuándo usarías cada diseño? (piloto · subgrupos · conclusión)

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

Figura 5. Resumen de las estimaciones muestrales obtenidas en R y mensajes de carga de la biblioteca dplyr.

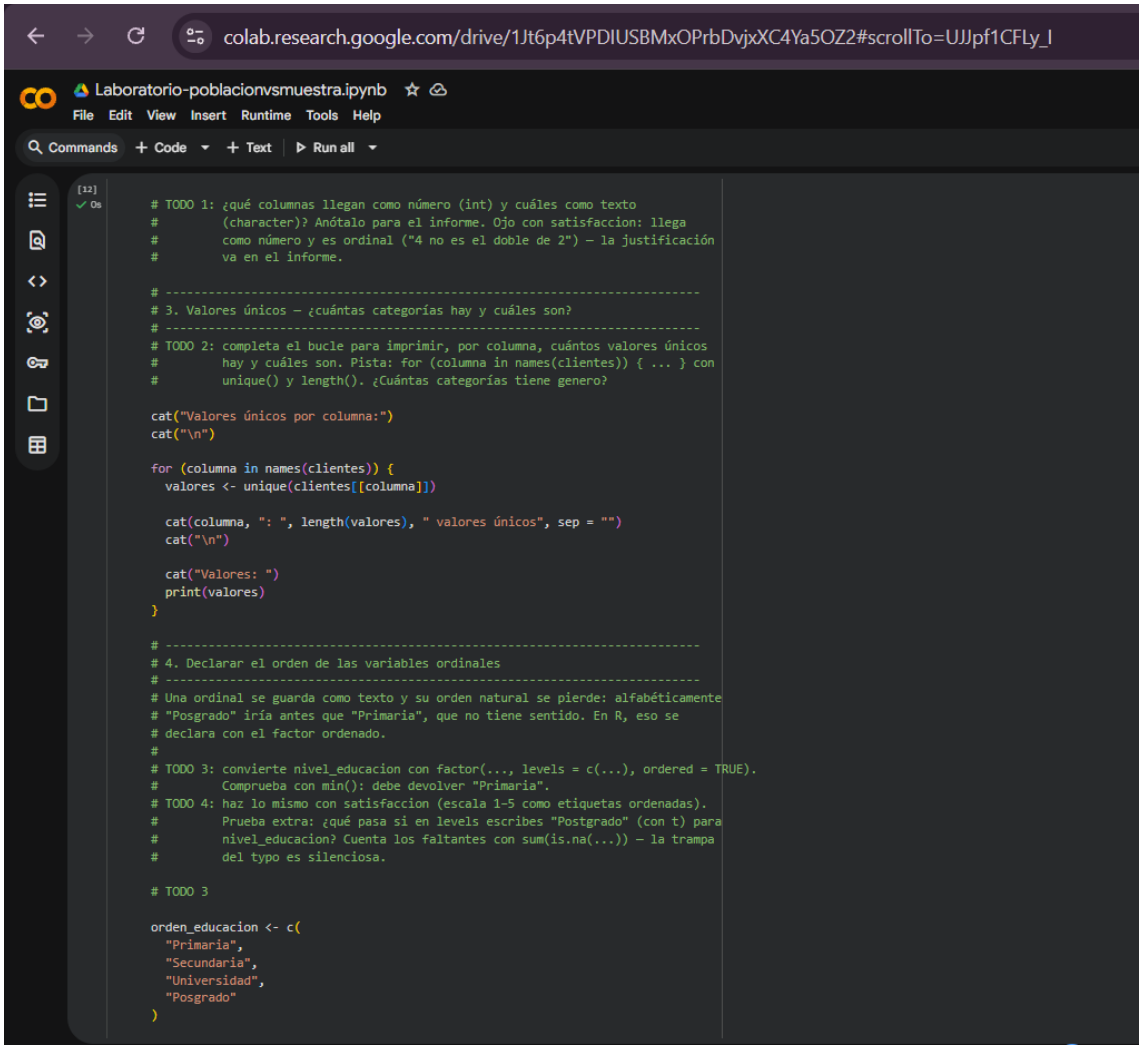


Figura 6. Código en R para identificar los valores únicos de las variables y establecer el orden de los niveles educativos.

[12]
✓ 0s

```r
clientes$nivel_educacion <- factor(
  clientes$nivel_educacion,
  levels = orden_educacion,
  ordered = TRUE
)

cat("Nivel mínimo de educación:")
cat("\n")
print(min(clientes$nivel_educacion))
cat("\n")

# TODO 4

clientes$satisfaccion <- factor(
  clientes$satisfaccion,
  levels = c(1, 2, 3, 4, 5),
  ordered = TRUE
)

cat("Satisfacción mínima:")
cat("\n")
print(min(clientes$satisfaccion))
cat("\n")

# Prueba extra

orden_incorrecto <- c(
  "Primaria",
  "Secundaria",
  "Universidad",
  "Postgrado"
)

educacion_error <- factor(
  as.character(clientes$nivel_educacion),
  levels = orden_incorrecto,
  ordered = TRUE
)

cat("Valores faltantes por error de escritura:")
cat("\n")
print(sum(is.na(educacion_error)))
cat("\n")
```

Figura 7. Conversión de las variables nivel educativo y satisfacción en factores ordenados, junto con la prueba de errores de escritura.

```
Dimensiones: 12 filas × 8 columnas
   cliente_id edad numero_hijos    genero nivel_educacion    ciudad
1        C001   34            2  Femenino     Universidad     Quito
2        C002   27            0 Masculino      Secundaria Guayaquil
3        C003   41            3  Femenino        Posgrado    Cuenca
4        C004   22            0 Masculino     Universidad     Quito
5        C005   36            1  Femenino      Secundaria Guayaquil
6        C006   29            2      Otro     Universidad     Quito
7        C007   48            4 Masculino        Primaria    Cuenca
8        C008   33            1  Femenino     Universidad Guayaquil
9        C009   25            0 Masculino      Secundaria     Quito
10       C010   39            2  Femenino        Posgrado Guayaquil
11       C011   31            1      Otro     Universidad    Cuenca
12       C012   45            3 Masculino      Secundaria     Quito
   ingresos_mensuales satisfaccion
1                1450            4
2                 980            3
3                2200            5
4                 760            2
5                1150            3
6                1350            4
7                 890            2
8                1680            4
9                 840            3
10               2450            5
11               1210            4
12               1070            2
Tipos inferidos en R:
'data.frame':   12 obs. of  8 variables:
 $ cliente_id        : chr  "C001" "C002" "C003" "C004" ...
 $ edad              : int  34 27 41 22 36 29 48 33 25 39 ...
 $ numero_hijos      : int  2 0 3 0 1 2 4 1 0 2 ...
 $ genero            : chr  "Femenino" "Masculino" "Femenino" "Masculino" ...
 $ nivel_educacion   : chr  "Universidad" "Secundaria" "Posgrado" "Universidad" ...
 $ ciudad            : chr  "Quito" "Guayaquil" "Cuenca" "Quito" ...
 $ ingresos_mensuales: int  1450 980 2200 760 1150 1350 890 1680 840 2450 ...
 $ satisfaccion      : int  4 3 5 2 3 4 2 4 3 5 ...
Valores únicos por columna:
cliente_id: 12 valores únicos
Valores:  [1] "C001" "C002" "C003" "C004" "C005" "C006" "C007" "C008" "C009" "C010"
[11] "C011" "C012"
edad: 12 valores únicos
Valores:  [1] 34 27 41 22 36 29 48 33 25 39 31 45
numero_hijos: 5 valores únicos
Valores: [1] 2 0 3 1 4
genero: 3 valores únicos
Valores: [1] "Femenino"  "Masculino" "Otro"
nivel_educacion: 4 valores únicos
Valores: [1] "Universidad" "Secundaria"  "Posgrado"    "Primaria"
ciudad: 3 valores únicos
Valores: [1] "Quito"     "Guayaquil" "Cuenca"
ingresos_mensuales: 12 valores únicos
Valores:  [1] 1450  980 2200  760 1150 1350  890 1680  840 2450 1210 1070
```

Figura 8. Carga de los 12 clientes en R, identificación de los tipos de datos y verificación de los valores únicos.

```
satisfaccion: 4 valores únicos
Valores: [1] 4 3 5 2
Nivel mínimo de educación:
[1] Primaria
Levels: Primaria < Secundaria < Universidad < Posgrado

Satisfacción mínima:
[1] 2
Levels: 1 < 2 < 3 < 4 < 5

Valores faltantes por error de escritura:
[1] 2


Resumen de la clasificación (para el informe):
  cliente_id         Categórica nominal (identificador)
  edad               Numérica discreta
  numero_hijos       Numérica discreta
  genero             Categórica nominal
  nivel_educacion    Categórica ordinal
  ciudad             Categórica nominal
  ingresos_mensuales Numérica continua
  satisfaccion       Categórica ordinal

Medias con semillas del 1 al 5:
Semilla 1: 1318.00 USD
Semilla 2: 1287.50 USD
Semilla 3: 1233.00 USD
Semilla 4: 1385.50 USD
Semilla 5: 1283.00 USD
```

Figura 9. Resultados de la clasificación en R, comprobación de los valores mínimos, detección de dos valores faltantes y medias obtenidas con cinco semillas.

```r
# ------------------------------------------------------------------------
# 5. Resumen de la clasificación (para el informe)
# ------------------------------------------------------------------------
# TODO 5: completa el resumen con el tipo de cada variable (el criterio que
#         justifica cada clasificación va en el informe).

clasificacion <- c(
  cliente_id         = "Categórica nominal (identificador)",
  edad               = "Numérica discreta",
  numero_hijos       = "Numérica discreta",
  genero             = "Categórica nominal",
  nivel_educacion    = "Categórica ordinal",
  ciudad             = "Categórica nominal",
  ingresos_mensuales = "Numérica continua",
  satisfaccion       = "Categórica ordinal"
)

cat("\nResumen de la clasificación (para el informe):")
cat("\n")

for (variable in names(clasificacion)) {
  cat(sprintf("  %-18s %s", variable, clasificacion[[variable]]))
  cat("\n")
}


cat("\nMedias con semillas del 1 al 5:\n")

for (semilla in 1:5) {
  set.seed(semilla)
  muestra <- poblacion[sample(nrow(poblacion), 20), ]
  media <- mean(muestra$ingresos_mensuales)

  cat(sprintf("Semilla %d: %.2f USD\n", semilla, media))
}
```

Figura 10. Código en R para resumir la clasificación estadística de las variables y calcular las medias de cinco sorteos aleatorios.

Python

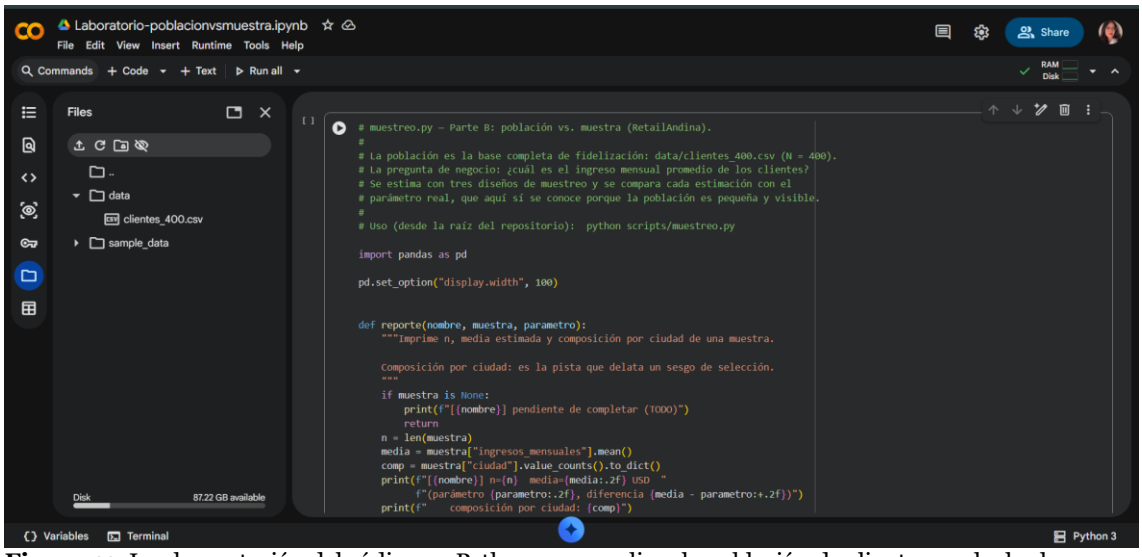


Figura 11. Implementación del código en Python para analizar la población de clientes y calcular las estimaciones muestrales.

```
# ------------------------------------------------------------------------
# 2. Muestra por conveniencia: "los primeros que llegaron"
# ------------------------------------------------------------------------
# TODO 1: toma como muestra las primeras 20 filas de la población.
#         Pista: .head(20) — igual que la demo de clase.
#         Después de correrlo, mira la composición por ciudad que imprime el
#         reporte: ¿es un corte al azar? ¿alguna ciudad falta por completo?
muestra_conv = poblacion.head(20)
reporte("Conveniencia", muestra_conv, parametro)
```

Figura 12. Selección de los primeros 20 clientes mediante el método de muestreo por conveniencia en Python.

```
# ------------------------------------------------------------------------
# 3. Muestra aleatoria simple: el sorteo de 20
# ------------------------------------------------------------------------
# TODO 2: sortea 20 clientes con la misma probabilidad para todos (20/400 = 5 %).
#         Pista: .sample(20, random_state=1). Con random_state el resultado es
#         reproducible; sin él cambia en cada ejecución.
# TODO 3: repite el sorteo con las semillas 2 a 5 y anota las 5 medias en el
#         informe: la variabilidad de la estimación es parte de la lección.
muestra_aas = poblacion.sample(20, random_state=1)
```

Figura 13. Implementación del muestreo aleatorio simple para seleccionar 20 clientes de forma reproducible.

```python
# ---------------------------------------------------------------------------
# 4. Muestra estratificada: 5 % dentro de cada ciudad
# ---------------------------------------------------------------------------
# TODO 4: muestrea dentro de cada ciudad (el estrato), con afijación
#         proporcional: 5 % de cada una → 10 de Quito, 6 de Guayaquil y 4 de
#         Cuenca (20 en total).
#         Pista: .groupby("ciudad") y .sample(frac=0.05, random_state=1).
#         ¿Qué garantiza este diseño que el sorteo simple no garantiza?
muestra_est = poblacion.groupby("ciudad").sample(frac=0.05, random_state=1)
```

Figura 14. Aplicación del muestreo estratificado, seleccionando el 5 % de los clientes de cada ciudad.

```
    Población: 400 clientes × 8 columnas
...  Parámetro poblacional — ingreso medio: 1330.72 USD
    Ciudades en la población: {'Quito': 200, 'Guayaquil': 120, 'Cuenca': 80}

    [Conveniencia] n=20  media=1348.50 USD  (parámetro 1330.72, diferencia +17.78)
        composición por ciudad: {'Quito': 20}
    [Aleatorio simple] n=20  media=1336.00 USD  (parámetro 1330.72, diferencia +5.28)
        composición por ciudad: {'Cuenca': 8, 'Quito': 8, 'Guayaquil': 4}
    [Estratificado] n=20  media=1287.50 USD  (parámetro 1330.72, diferencia -43.22)
        composición por ciudad: {'Quito': 10, 'Guayaquil': 6, 'Cuenca': 4}

    Parámetro poblacional: 1330.72 USD
                           diseño  n  media estimada
                Conveniencia (head) 20         1348.5
             Aleatorio simple (sample) 20         1336.0
    Estratificado (groupby + frac) 20         1287.5

    Con estas salidas responde en informe.md las preguntas P2 a P5:
      P2: ¿cuál diseño quedó más cerca del parámetro y por qué un solo
          número no basta para declarar un ganador?
      P3: ¿qué defecto tiene la muestra por conveniencia y cómo lo delata
          su composición por ciudad? ¿Se corrige con más filas?
      P4: ¿qué implica la variabilidad entre semillas para una decisión
          tomada con un solo estudio?
      P5: ¿cuándo usarías cada diseño? (piloto · subgrupos · conclusión)
```

Figura 15. Resultados del muestreo en Python, con el parámetro poblacional de 1330,72 USD y las medias estimadas mediante los tres diseños.

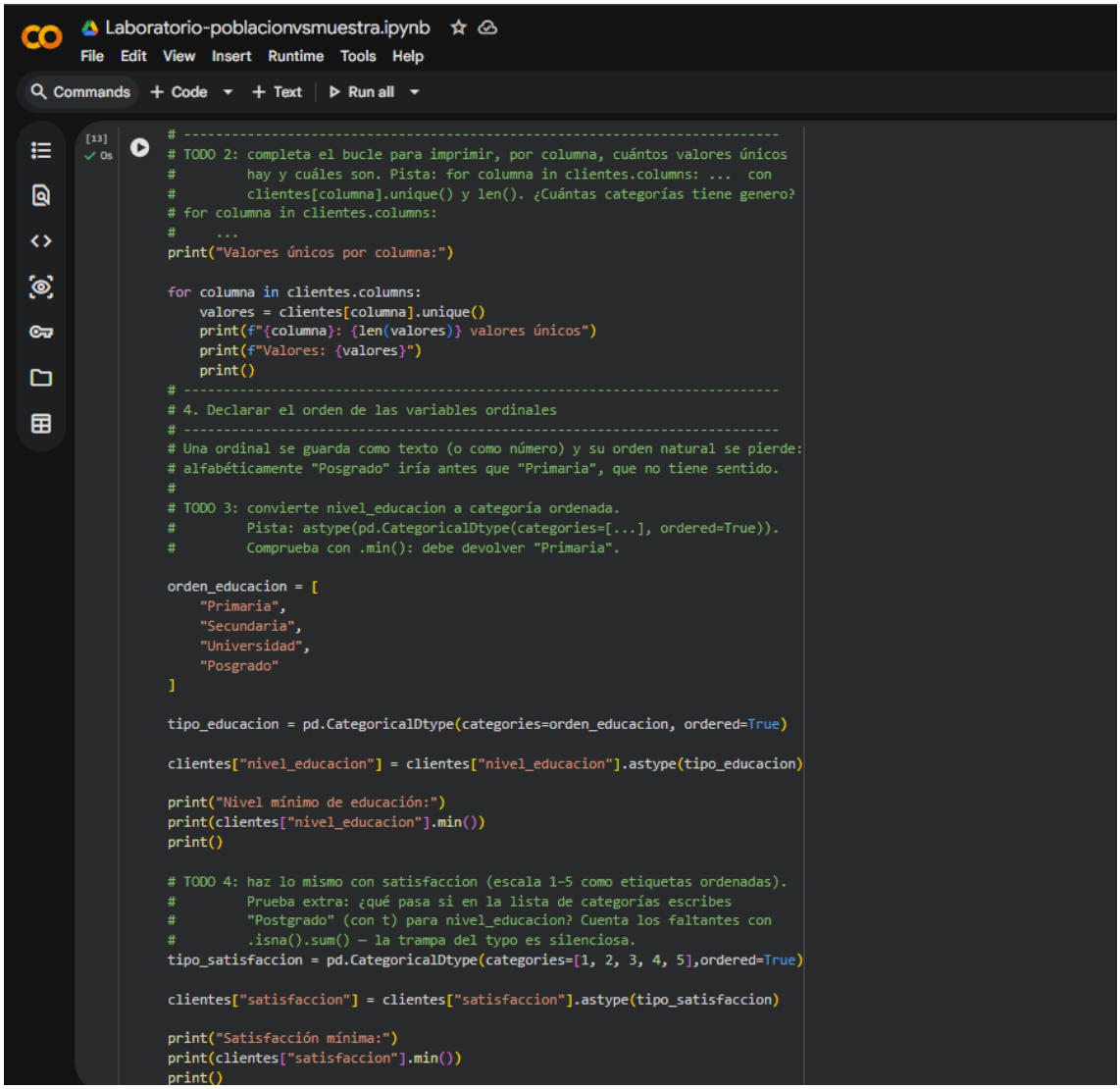


Figura 16. Código en Python para identificar valores únicos y convertir nivel educativo y satisfacción en categorías ordinales.

```python
# Prueba extra
orden_incorrecto = [
    "Primaria",
    "Secundaria",
    "Universidad",
    "Postgrado"
]

tipo_incorrecto = pd.CategoricalDtype(categories=orden_incorrecto,ordered=True)

valores_invalidos = ~clientes["nivel_educacion"].isin(
    orden_incorrecto
)

print("Valores faltantes por error de escritura:")
print(valores_invalidos.sum())
print()


# ------------------------------------------------------------------------
# 5. Resumen de la clasificación (para el informe)
# ------------------------------------------------------------------------
# TODO 5: completa el resumen con el tipo de cada variable (el criterio que
#         justifica cada clasificación va en el informe).
clasificacion = {
    "cliente_id": "Categórica nominal (identificador)",
    "edad": "Numérica discreta",
    "numero_hijos": "Numérica discreta",
    "genero": "Categórica nominal",
    "nivel_educacion": "Categórica ordinal",
    "ciudad": "Categórica nominal",
    "ingresos_mensuales": "Numérica continua",
    "satisfaccion": "Categórica ordinal",
}
print("Resumen de la clasificación (para el informe):")
for variable, tipo in clasificacion.items():
    print(f"  {variable:20s} {tipo}")
```

Figura 17. Implementación de la prueba de errores de escritura y elaboración del resumen de clasificación de variables en Python.

```
Dimensiones: 12 filas × 8 columnas

Primeras filas:
cliente_id  edad  numero_hijos    genero nivel_educacion    ciudad  ingresos_mensuales  satisfaccion
      C001    34             2  Femenino     Universidad     Quito                1450             4
      C002    27             0 Masculino      Secundaria Guayaquil                 980             3
      C003    41             3  Femenino        Posgrado    Cuenca                2200             5
      C004    22             0 Masculino     Universidad     Quito                 760             2
      C005    36             1  Femenino      Secundaria Guayaquil                1150             3
      C006    29             2      Otro     Universidad     Quito                1350             4
      C007    48             4 Masculino        Primaria    Cuenca                 890             2
      C008    33             1  Femenino     Universidad Guayaquil                1680             4
      C009    25             0 Masculino      Secundaria     Quito                 840             3
      C010    39             2  Femenino        Posgrado Guayaquil                2450             5
      C011    31             1      Otro     Universidad    Cuenca                1210             4
      C012    45             3 Masculino      Secundaria     Quito                1070             2

Tipos inferidos (dtype):
cliente_id            object
edad                   int64
numero_hijos           int64
genero                object
nivel_educacion       object
ciudad                object
ingresos_mensuales     int64
satisfaccion           int64
dtype: object

Valores únicos por columna:
cliente_id: 12 valores únicos
Valores: ['C001' 'C002' 'C003' 'C004' 'C005' 'C006' 'C007' 'C008' 'C009' 'C010'
 'C011' 'C012']

edad: 12 valores únicos
Valores: [34 27 41 22 36 29 48 33 25 39 31 45]

numero_hijos: 5 valores únicos
Valores: [2 0 3 1 4]

genero: 3 valores únicos
Valores: ['Femenino' 'Masculino' 'Otro']

nivel_educacion: 4 valores únicos
Valores: ['Universidad' 'Secundaria' 'Posgrado' 'Primaria']

ciudad: 3 valores únicos
Valores: ['Quito' 'Guayaquil' 'Cuenca']

ingresos_mensuales: 12 valores únicos
Valores: [1450  980 2200  760 1150 1350  890 1680  840 2450 1210 1070]

satisfaccion: 4 valores únicos
Valores: [4 3 5 2]
```

Figura 18. Visualización de los 12 registros de clientes en Python, tipos de datos inferidos y valores únicos de cada columna.

```
Nivel mínimo de educación:
Primaria

Satisfacción mínima:
2

Valores faltantes por error de escritura:
2

Resumen de la clasificación (para el informe):
  cliente_id           Categórica nominal (identificador)
  edad                 Numérica discreta
  numero_hijos         Numérica discreta
  genero               Categórica nominal
  nivel_educacion      Categórica ordinal
  ciudad               Categórica nominal
  ingresos_mensuales   Numérica continua
  satisfaccion         Categórica ordinal
```

Figura 19. Resultados finales de la clasificación en Python, verificación del orden de las variables ordinales y detección de dos valores faltantes.